AgentProf: Semantic Profiling for AI Agents

Abstract

AI agents orchestrate multi-step activities across hundreds of sessions, but existing observability tools provide only per-session debugging views with no cross-session profiling layer. To improve quality, safety, and cost efficiency, developers need aggregate analysis of which task categories consume the most budget, where failures concentrate, and which workflows trigger unsafe side-effects. Systems observability calls this aggregate analysis *profiling*, and agent observability today has only the debugging layer. Existing tools organize events into per-session span trees or cluster inputs by topic, but none provides cross-session resource attribution by semantic category. The flame-graph profiling methodology that answers these questions for traditional software does not directly apply because agent traces lack deterministic identifiers and runtime call-stack hierarchies. We propose a *semantic operation stack model*. Uniform *operations* represent all agent activities, and *operation stacks* replace the runtime call stack with query-time field projections that fold identical frame sequences. AgentProf implements this model as a profiler with pluggable algorithms for *intent attribution* (deriving deterministic labels from natural-language fields) and *stack construction* (building aggregation hierarchies from available fields), compiling local agent traces into pprof-compatible profiles. On 325 real agent sessions, semantic profiling separates over 90% of system-effect weight that remains mixed under session-level views. On four public datasets with hidden ground-truth labels (34,539 operations), profiler-ranked groups localize labeled problems at 9.4% inspection work versus flat summaries, with 45% fewer groups than fixed-session drilldown.

1 Introduction

AI agents orchestrate multi-step activities that span two layers, high-level intent (prompts, LLM calls, tool invocations) and low-level system effects (process spawns, file and network I/O). As agents evolve from short, scripted workflows into complex systems that run for hours, production teams accumulate hundreds of sessions over weeks or months.

To improve agent quality, safety, and cost efficiency, developers need to analyze operational behavior across sessions. Which categories of tasks consume the most token budget? Where do failures concentrate across workflows? Which behavioral patterns trigger unsafe system effects? Systems observability has long distinguished this kind of aggregate analysis (profiling) from single-execution debugging. Agent observability today has only the debugging layer.

Existing agent observability tools do not fill this gap. LangSmith [13], Langfuse [14], Phoenix [1], and OpenTelemetry GenAI [24] organize events into per-session span trees, effective for diagnosing a single run but unable to aggregate across sessions by semantic category. Datadog LLM Observability [6] clusters user messages by topic, and Laminar [12] extracts structured signals from traces, but both characterize *input distributions* ("30% of users ask code questions") rather than *resource attribution* ("review tasks consume 40% of the token budget"), because labels at the request level do not propagate to downstream tool calls and system effects.

The profiling methodology is well-established for traditional software. Flame graphs [11] fold identical call stacks into a single chart where width represents cost. Two properties make this work. Function names are deterministic, so identical code paths always produce the same stack frames, and runtime call nesting provides the aggregation hierarchy. Agent traces break both (§2). First, the core unit of agent work is a natural-language prompt, and two prompts expressing the same intent share no common string. Framework-level span names (`LLMCall`, `ToolInvoke`) describe mechanisms, not semantic intent. Second, agent events have no runtime call-stack hierarchy because prompts, tool calls, and system effects are not nested by execution the way function calls produce stack frames.

Despite these differences, the fundamental profiling methodology (define units, assign stable labels, construct aggregation hierarchies, fold identical stacks) transfers to agent traces. We propose a *semantic operation stack model* with two components. *Operations* are uniform records with string fields and additive measures that represent all agent activities (prompts, LLM calls, tool invocations, and system effects) without type-specific objects. *Operation stacks* are ordered lists of fields chosen at query time. Each operation is projected to a frame sequence, and operations with identical sequences merge, producing the folded-stack representation that flame graphs visualize. Different field selections answer different questions (by task category, by execution phase, by session, or by action type) from the same data.

We implement this model in AgentProf, an offline profiler that compiles local agent traces into pprof-compatible profiles (Figure 1). Two pluggable algorithm frameworks enrich the model. *Intent attribution* derives deterministic, low-cardinality labels from natural-language fields via regex rules, a local LLM tagger, or unsupervised clustering. This restores the folding precondition that CPU profiling obtains for free from function names. *Stack construction* builds aggregation hierarchies from available fields, either from a user-specified ordered list of fields or via automatic induction that scores semantic shifts and partition coherence. On 325 real Codex and Claude sessions (183,714 system observations), semantic profiling separates over 90% of system-effect weight that remains mixed under session-level views. On four public datasets with hidden ground-truth labels (34,539 operations), profiler-ranked groups localize labeled problems at 9.4% inspection work versus flat summaries, with 45% fewer groups than fixed-session drilldown. Semantic-label quality exceeds V-measure 0.7 on 7 of 9 held-out datasets.

This paper makes three contributions:

1. **Semantic operation stack model.** We identify the missing profiling layer in agent observability and propose a model of uniform operations and query-time operation stacks (§2–§3).
2. **System: AgentProf.** A profiler that implements this model with pluggable intent attribution and stack construction, producing pprof-compatible output (§4).
3. **Hidden-label evaluation.** We evaluate profiler output against hidden ground-truth trajectory labels, analogous to injecting

known bottlenecks in CPU profiler evaluation, on 325 real sessions and 4 public datasets (§5).

2 Background and Motivation

2.1 AI Agent Observability

As introduced in §1, agent activities span both intent and system-effect layers. Existing observability tools cover these layers separately. LangSmith [13], Langfuse [14], Phoenix [1], and OpenTelemetry GenAI [24] organize agent-level events into per-execution *span trees*. System monitors show process and file activity without semantic attribution. The span-tree design is effective for debugging. A developer can locate a failing span, inspect its attributes, and trace its parent chain. AgentSight [34] bridges the two layers by correlating eBPF-captured system events with intercepted LLM traffic, but the resulting traces still lack aggregate profiling.

2.2 Profiling vs. Debugging

Debugging inspects a single execution, but profiling aggregates across executions to answer questions like "where does the budget go?" CPU profilers solved this aggregation problem decades ago. A flame graph [11] represents a profile as weighted *samples*, each attached to a *call stack*. Identical stacks merge, and width represents aggregate cost. The key enabler is *deterministic* function names, so the same code path always produces the same frames.

Agent activities break this precondition in two ways. First, prompts are natural language and lack deterministic identifiers. Second, agent events have no inherent call stack because they lack the runtime-determined parent-child nesting that function calls provide. Pprof's `tagroot`/`tagleaf` options add label-derived pseudo-frames to an existing execution stack, but agent activities have no execution stack to augment.

2.3 The Aggregation Gap

Existing agent observability tools are designed for debugging, not profiling, and this creates three concrete gaps.

First, sessions are boundaries, not categories. A single session may contain review, debugging, and refactoring phases, and splitting by session assigns all of them to the same bucket. Two sessions performing the same task produce separate buckets that cannot be merged without external logic.

Second, span names describe mechanisms (`LLMCall`, `ToolInvoke`, `ChatCompletion`), not user intent (`review`, `debug`, `test`). Grouping by span name tells you how many LLM calls occurred, not which categories of work consumed the budget.

Third, these limitations compound in practice. A common system effect like running a test suite may appear thousands of times across sessions, but a flat summary shows only a single counter with no way to attribute executions to the semantic intent that triggered them. We quantify this in RQ1 (§5).

These observations yield three design requirements:

- **R1: Uniform record type.** Agent activities mix prompts, LLM calls, tool invocations, GUI actions, process events, and benchmark labels. The profiler needs a single record type that can represent all of them without type-specific objects.
- **R2: Query-time aggregation.** The same data must support multiple aggregation shapes (by task, phase, action, session, or dataset) without rebuilding the input. Sessions and spans must be fields, not fixed hierarchy nodes.
- **R3: Deterministic labels for folding.** Free-form prompts must be mapped to short, stable, repeatable labels before folding. This *intent attribution* step has no analogue in CPU profiling, where function names are already deterministic.

3 Design

The three requirements (a uniform record type, query-time aggregation, and deterministic labels for folding) drive the design. The semantic operation stack model has two components, operations (§3.1) and operation stacks (§3.1), satisfying R1 and R2. Two pluggable algorithm frameworks enrich the model. Intent attribution (§3.2) satisfies R3 by deriving deterministic labels, and stack construction (§3.1) builds aggregation hierarchies from available fields. The profiling pipeline has five stages: (1) parse raw traces into operations, (2) enrich operations via intent attribution or mapping rules, (3) filter by a user-specified predicate, (4) project each operation onto a stack frame sequence chosen at query time, and (5) fold operations with identical stacks, summing their weights.

3.1 Semantic Operation Stack Model

Because agent activities span both the intent and system-effect layers described in §2, a profiler should not distinguish between them at the object-model level. AgentProf therefore represents every activity as a single abstraction, the `operation`, a weighted record with string fields and additive measures. Each operation carries string fields (`project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`, `domain`, `status`, ...) and additive measures (token count, duration, occurrence count). A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event are all operation records that differ only in which fields are populated.

An `operation stack` is an ordered list of fields chosen at query time. Given a stack specification $[f_1, f_2, \ldots, f_k]$, each operation o is projected to a frame sequence $\langle o.f_1, o.f_2, \ldots, o.f_k \rangle$. Operations with identical frame sequences are merged and their weights are summed. Unlike CPU profiling, where the call stack is determined by execution, the stack is determined by the query. The same operations can be folded as a flat task summary, a fixed-session tree, a semantic phase/action hierarchy, or a dataset-native trajectory, and changing the fields changes the aggregation without changing the data. Sessions and spans are optional operation fields, not structural skeleton nodes, so the same data supports both debugging views (include `session`) and aggregate profiling views (exclude it). A *view* is a triple (φ, σ, w): a predicate φ selects which operations participate, a stack function $\sigma = [f_1, \ldots, f_k]$ maps each operation to its frame sequence, and a weight function w assigns a nonnegative measure. Four built-in defaults (Table 1) answer common questions.

3.2 Algorithms

Intent attribution. Intent attribution maps free-form prompts to short, stable, deterministic labels, restoring the folding precondition that agent prompts violate (§2). The framework supports three pluggable backends. *Regex rules* are the production default: rules

Table 1: Built-in views. Each shares the same operation set but selects different activities, stack structures, and weights.

View	φ (select)	w (width)	Question
`tokens`	LLM calls	token count	Budget distribution?
`time`	All timed ops	duration (s)	Time distribution?
`files`	File-effect ops	event count	Repo coverage?
`network`	Net-effect ops	event count	External contacts?

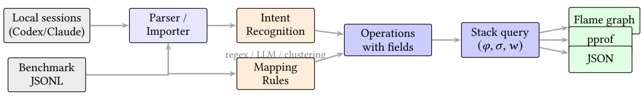

Figure 1: System architecture. Local agent histories enter through `agent-session` parsing. Public datasets enter as operation JSONL. Both paths produce operations. The stack query and output are identical.

have the form `KIND:TAG=REGEX`, are evaluated in order (first match wins), and the developer iterates with AgentProf until the unmatched rate drops below 5%, typically in 5–10 rounds. A *local LLM tagger* (e.g., a quantized 3B-parameter model via llama.cpp) generates a single-word label per prompt with grammar-constrained decoding (forcing output to match a predefined token grammar) and caching. *Unsupervised clustering* via TF-IDF and K-Means discovers natural prompt groups without predefined rules, feeding rule authoring. For structured traces where action types and quality annotations are explicit fields, *mapping rules* derive new fields from existing ones (e.g., `phase:navigate=type:(click|scroll|key)`). All backends are pluggable, and a *profiling configuration* records the complete analysis configuration for deterministic replay.

Stack construction. Stack construction builds aggregation hierarchies from available fields. When users supply a ordered list of fields, the projection is direct. When no chain is supplied, AgentProf can *induce* operation stacks automatically by scoring adjacent operations for semantic shifts, field changes, and partition coherence, then cutting at boundaries where the evidence is strongest. The induced stacks have variable depth, and a configurable depth cap controls the granularity. This mode produces coarser groups than hand-specified stacks but requires no domain knowledge, making it useful for initial exploration (§5).

4 Implementation

AgentProf is an offline, post-hoc profiler implemented as a Rust CLI (Figure 1). It reads recorded traces, not live events. The `agent-session` crate reads Codex and Claude Code local JSONL history files and reconstructs prompts, LLM calls, tool invocations, and their triggered file and network effects. Each prompt opens a span. LLM calls, tool calls, and system effects within that span inherit its semantic label, connecting intent to system behavior without runtime instrumentation. Public benchmark datasets enter as operation JSONL with mapping rules deriving fields before stack construction. Standard Chrome Trace Event format can also be imported, with non-AgentProf protocol fields preserved as operation fields.

Output formats include pprof protobuf (compatible with `go tool pprof -http`), folded-stack text (compatible with `flamegraph.pl`), standalone SVG flame graphs, and JSON summaries. Operations can also be exported as standard Chrome Trace Event JSON and reimported with byte-identical folded output, enabling interoperability with trace viewers such as Perfetto. All outputs are deterministic given the same input and profiling configuration.

AgentProf is the offline profiling component of the AgentSight [34] observability framework. AgentSight provides real-time eBPF-based capture of SSL traffic, process events, and file/network effects. AgentProf aggregates the recorded traces into semantic profiles, so the two are complementary: AgentSight answers "what happened during this run" (debugging), and AgentProf answers "where did the budget go across runs" (profiling).

5 Evaluation

We use two classes of data. *Real agent sessions*: 325 readable local sessions (198 Codex, 50 Claude, 77 Claude subagent) from a multi-month development project, containing 183,714 system observations. *Public labeled traces*: 15 families covering web, mobile, desktop, software, API, and dialogue agents (Table 2), totaling 47,590 operations through mapping rules. All public traces are real agent or human executions collected and labeled by their respective benchmark projects. Four oracle-rich families (those with ground-truth quality annotations suitable for evaluation: AgentRewardBench, SATraj-OS, AgentNet, OSWorld-Human) provide hidden labels for RQ2. We measure semantic separation via the mixed-weight percentage defined in RQ1. For localization quality (RQ2), we use average precision (AP, the area under the precision-recall curve when groups are ranked by suspected problem density), recall at various inspection budgets, and work-to-first-positive (the fraction of groups inspected before finding one positive). For label agreement (RQ3), we use V-measure (a clustering-agreement score ranging from 0 to 1) and boundary F1.

5.1 RQ1: Does Semantic Profiling Improve Resource Attribution?

The strongest argument against semantic profiling is that traditional tools "can already see the same information." To test this, we run a semantic-axis ablation on 325 real Codex/Claude sessions (183,714 system observations). We progressively add label axes (no labels, session only, prompt only, both) and measure the *mixed-weight percentage*, the fraction of system-effect weight in buckets that mix observations from multiple task categories. A lower mixed-weight percentage means the profiler correctly separates effects by task category. The residual percentage measures weight in buckets where the majority category accounts for less than 90% of the bucket, capturing partial mixing.

Table 3 shows the result on 325 real Codex/Claude sessions. Without semantic labels, 90.4% of system-effect weight falls into mixed buckets. A flat `cargo test` counter shows 2,903 executions but cannot distinguish whether they came from `review`, `debug`, `refactor`, or `test` prompts. Adding only the prompt label reduces mixed weight to 36.7% and residual to 7.5%. The session label alone barely helps (84.4%), because sessions typically contain multiple intent phases.

Table 2: Public labeled trace families. All traces are real agent or human executions. The bottom four rows provide oracle labels for RQ2.

Family	Native labels	Access	Trace origin	Eval. role
WebLINX [20], Mind2Web [7], Agent-Trek [30], WebShop [32]	Web actions, tasks, rewards	HF / public	Real web agents	Coverage
API-Bank [15], Tool-Bench [25], tau-bench [33]	API calls, outcomes	HF / JSONL	Real API agents	Coverage
SWE-agent [31]	Issue trajectories	HF viewer	Real coding agent	Coverage
AndroidCtrl [16], GUI-Odyssey [19], Scale-CUA [18]	Mobile/GUI actions	Public/HF	Real mobile agents	Coverage
AgentRewardBench [21]	Success, side-effect, looping	HF annotations	Real BrowserGym agents	RQ2 tasks
SATraj-OS [5]	Safety, reward, attack type	HF config	Real desktop agents	RQ2 task
OSWorld-Human [29]	Human grouped-action traces	GitHub JSON	Real human OS use	RQ2 task
AgentNet [27]	Step correctness, redundancy	HF JSONL	Real desktop agents	RQ2 tasks

Table 3: Semantic-axis ablation on 183,714 system observations from 325 real sessions. All rows preserve the same total weight.

Projection	Unique stacks	Mixed weight %	Residual %
No semantic labels	11,967	90.4	44.7
Session label only	15,027	84.4	33.4
Prompt label only	24,703	36.7	7.5
Session + prompt	26,829	0.0	0.0

A label-permutation test (1,000 shuffles, $p = 0.001$) confirms the separation is not random. Semantic-label quality is validated in RQ3.

Beyond label separation, we test whether choosing different field selections for the operation stack produces views that flat grouping cannot. We take the same 13,265 operations from the four oracle-rich families and fold them with five different field selections, counting how many distinct groups each produces. The same operations fold into 9 dataset groups, 57 phase groups, 226 semantic groups, 455 action groups, or 3,757 fixed-session groups by changing only the stack fields. A flat GROUP BY on a single label cannot produce these views because different stack depths answer different questions (task-level budget vs. action-level duplication vs. session-level drilldown). The same model also covers all 15 public trace families (Table 2) without type-specific objects.

Different views also produce different rankings. The `time` and `tokens` views of the same 325 sessions share only 7/10 top prompt categories (Spearman $\rho = 0.623$). `network`-tagged prompts rank 8th by time (I/O wait) but 93rd by tokens, showing that the choice of weight function changes the analysis conclusions.

Finally, we test whether the profiler can produce useful groups without any user-specified fields. We run the automatic stack-induction mode (§3.1) on the same six tasks used in RQ2 and compare the resulting group count and localization quality against flat summaries and fixed-session drilldown. Induced stacks produce a median of 12 groups (versus 285 for fixed-session and 1 for flat), with 4/6 tasks generating variable-depth stacks. The induced stacks achieve lower AP than hand-configured stacks (median 0.276 vs 0.312) because they use only visible fields, but they reduce inspection work to 65.3% versus flat summaries without requiring any user configuration.

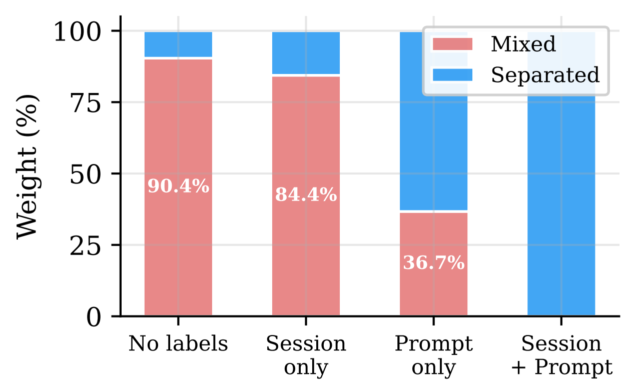

Figure 2: RQ1: Semantic-axis ablation on 183,714 system observations. Adding prompt labels reduces mixed weight from 90.4% to 36.7%. Session labels alone barely help (84.4%).

Figure 2 visualizes the ablation. Figure 3 shows the resulting flame graphs for the `tokens` and `files` views, illustrating that changing only the stack fields and weight function produces different aggregates from the same operations.

5.2 RQ2: Does Profiler Output Correspond to Real Problems?

RQ2 tests whether the profiler's top-ranked groups actually contain real failures, safety violations, or wasted effort. We use public benchmark datasets that provide ground-truth quality annotations (failure, safety violation, redundant step, human-task boundary), hide these annotations from the profiler, and check whether the top-ranked groups recover them, analogous to injecting known bottlenecks in a CPU profiler evaluation.

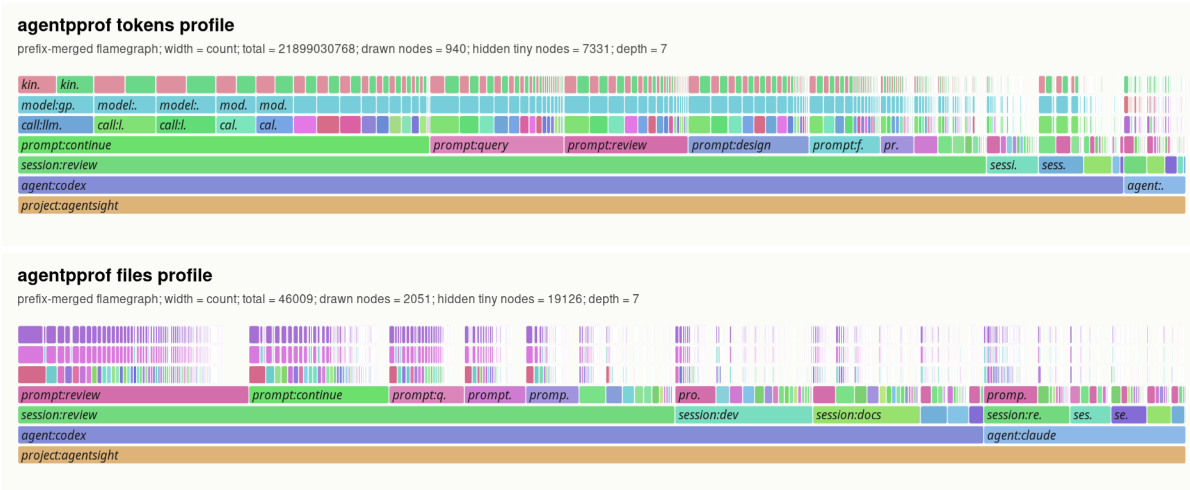

Figure 3: Flame graphs of the same 325 sessions under two different views. ***Top (tokens view):*** **width represents token count; bottom frames show project and agent, middle frames show prompt labels, top frames show LLM calls and models.** ***Bottom (files view):*** **width represents file-operation count; `prompt:review` and `prompt:continue` dominate file activity. Changing only the stack fields and weight function produces a different aggregate from the same operations (Table 1).**

The benchmark uses six tasks across four datasets (AgentRewardBench looping and side-effect, SATraj-OS unsafe operation, AgentNet incorrect and redundant step, and OSWorld-Human group start), totaling 34,539 operations and 3,699 positives. We compare six grouping strategies (flat summary, fixed-session drilldown, dataset-native hierarchy, raw-action grouping, operation-stack profiling, and an oracle that uses the hidden labels) and rank groups by the fraction of hidden positives they contain.

Table 4 summarizes the results. Flat summaries recover all positives but require inspecting everything (Work@5 = 1.0). Fixed-session drilldown finds a first positive quickly (WTFP = 0.004) but fragments the workload into 285 groups with only 2.3% top-five recall. Operation-stack profiling inspects 9.4% of the work in the top five groups, recovers 39.0% of positives within 30% of the inspection budget, and reduces groups to 157.5.

Figure 4 visualizes this tradeoff: operation-stack profiling sits in the low-work, moderate-recall region. The comparison also serves as evidence for the operation/operation-stack model (§3). Fixed-session drilldown and dataset-native hierarchy are alternative abstractions that fix the hierarchy at session or dataset boundaries. Operation stacks subsume both as special cases (include `session` in the stack to recover fixed-session; include `dataset` to recover dataset-native). On localization, operation stacks improve top-five recall over fixed-session on 5/6 tasks, reduce median group count by 45%, and save 90% of inspection work versus flat summaries. Fixed-session drilldown retains lower work-to-first-positive on 4/6 tasks, confirming that profilers and debuggers are complementary.

Statistical validation supports these comparisons. Bootstrap (10,000 resamples) and label-permutation (2,000 trials) confirm that the AP differences are stable and exceed the 95% confidence threshold on all six tasks.

We also test whether stack depth matters by sweeping the induced-stack depth cap from 1 to 5 on the same six tasks and measuring AP

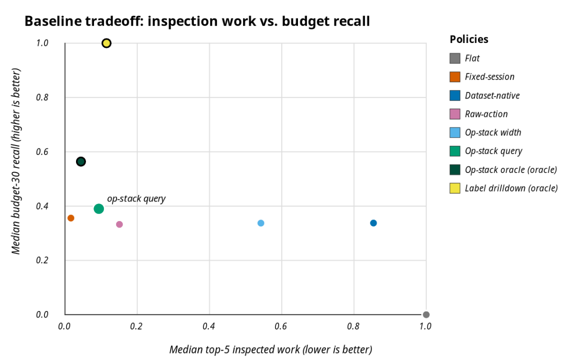

Figure 4: RQ2: Inspection work vs. recall at 30% inspection budget. Operation-stack profiling (labeled) achieves low work (0.094) with moderate recall (0.39), while flat requires full inspection and fixed-session fragments into 285 groups. Oracle policies (large dots) are upper bounds.

and inspection work at each depth. AP peaks at depth 3 (median 0.287) while inspection work is lowest at depth 5 (median 0.473). Different tasks achieve their best AP at different depths (2, 3, 4, and 5), confirming that depth should be tuned per objective rather than fixed globally.

The profiler's configuration is also actionable. Modifying stack fields, mapping rules, or ranking criteria and re-running on the same operations improves AP on 5/6 tasks (median AP improvement 0.038), confirming that the profiler exposes useful tuning knobs rather than requiring a fixed configuration.

Table 4: RQ2 hidden-label localization (medians across six tasks). AP = average precision. R@5 = recall in top 5 groups. Work@5 = fraction of total work in top 5 groups. R@30% = recall at 30% inspection budget. WTFP = work-to-first-positive (fraction inspected before finding one positive). Hidden = 1 means the policy uses hidden labels (oracle upper bound). "—" = group count varies by task.

Policy	Hidden	AP	R@5	F1@5	Work@5	R@30%	WTFP	Groups
Flat (all-in-one)	0	0.168	1.000	0.287	1.000	0.000	1.000	1.0
Per-session drilldown	0	0.348	0.023	0.043	0.016	0.356	0.004	285.0
Dataset-native hierarchy	0	0.357	0.867	0.282	0.854	0.338	0.058	—
Raw-action grouping	0	0.274	0.244	0.220	0.151	0.333	0.037	—
Operation-stack profiling	0	0.312	0.188	0.198	0.094	0.390	0.038	157.5
Op-stack + oracle fields	1	0.599	0.300	0.403	0.044	0.564	0.012	—
Oracle (hidden labels)	1	1.000	0.670	0.797	0.115	1.000	0.038	—

Table 5: Leave-dataset-out mapping quality. V-measure between mapping-derived `phase` and native `action` labels. 13,265 operations from the four oracle-rich families across 9 held-out evaluations.

Held-out dataset	Ops	V-measure	Boundary F1
mind2web	774	1.000	1.000
webshop-expert	2,504	1.000	1.000
swe-agent	496	0.926	0.962
weblinx-chat	500	0.872	0.860
agenttrek	200	0.862	—
gui-odyssey	7,868	0.811	0.842
android-control	9	0.716	0.727
toolbench	866	0.134	0.353
api-bank	48	0.000	—

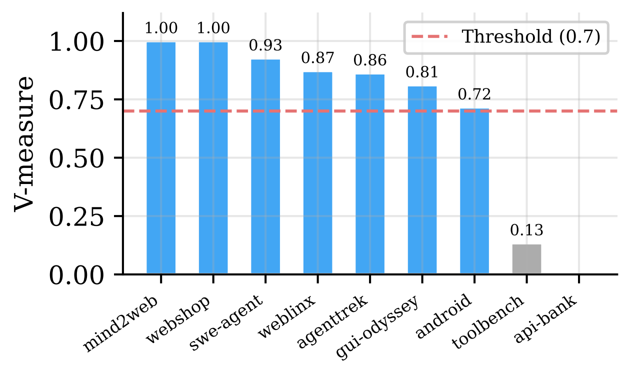

Figure 5: RQ3: Leave-dataset-out V-measure between mapping-derived `phase` and native `action` labels. The dashed line marks 0.7. Seven of nine datasets exceed this threshold.

5.3 RQ3: How Accurate Are the Labels?

RQ1 and RQ2 assume that semantic labels (the labels derived by intent attribution or mapping rules) are useful. RQ3 tests whether these semantic labels are accurate by comparing them against ground-truth annotations from the benchmark datasets. For free-form traces (real sessions), we measure whether the intent-attribution backends run reliably at scale and produce well-formed labels. Semantic accuracy of the LLM/regex tagger on free-form prompts remains future work because none of the available datasets (67,304 rows across 6 sources) simultaneously expose free-form text and ground-truth semantic labels, making same-input accuracy evaluation infeasible with current data.

On 325 real sessions, the LLM tagger processes 118,021 tag requests with zero failures, 100% syntax validity, and 70% cache hit rate (operational details in RQ4).

Public benchmark datasets provide native action labels that serve as ground truth for mapping quality. We run a leave-dataset-out evaluation. For each of 9 datasets, we train mapping rules on the other 8 and measure V-measure between the mapping-derived `phase` field and the dataset's native `action` labels, plus boundary F1 between mapped phase boundaries and native action boundaries.

Table 5 and Figure 5 show the results. On 7/9 datasets, V-measure exceeds 0.7, indicating strong agreement between mapping-derived and native labels. The two low-scoring datasets (toolbench, api-bank) have atypical action vocabularies that the held-out mapping rules do not cover.

Supervised boundary predictors beat simple baselines on 4/5 tested datasets (e.g., F1 0.77 vs 0.71 on OSWorld-Human), confirming that mapping rules produce useful labels but are not automatic intent discovery.

5.4 RQ4: What Is the Profiling Cost?

AgentProf is an offline, post-hoc profiler, so it adds zero overhead to the agent's runtime. The profiling cost is the time and resources spent after the session ends.

We re-execute 76 tracked profiling configurations twice (152 invocations) to measure cost and verify determinism. All 76 specs produce byte-identical output across both runs. Median profiling time is 1.6 s per spec (p95 2.8 s, max 4.3 s), and median output size is 37.5 KB. The slowest specs process views with over 600 unique stacks, while the 60 robustness-check specs all complete within 1.6 s. The full profile-spec composition path (operation files, query predicates, rank rules, and stack depth) stays deterministic across all 12 tested configuration variants.

Intent attribution adds a one-time cost for new prompts. Of 118,021 tag requests on 325 real sessions, 70% are cache hits (zero cost), and the remaining 35,136 new calls complete at p95 latency of 31 ms per label using a local 3B-parameter model. Grammar-constrained decoding ensures 100% syntax validity (2,700/2,700 test labels). Because labels are cached, re-profiling the same sessions with different stack fields or predicates incurs no additional LLM calls.

Limitations. The real-session data comes from a single multi-month development project, so the label distributions may not generalize to other domains. Semantic accuracy of the intent-attribution backends on free-form prompts remains unevaluated because no available dataset simultaneously exposes free-form text and ground-truth semantic labels (as discussed in RQ3 above). The evaluation uses fixed-session drilldown as a proxy for span-tree debugging tools, not a real OpenTelemetry or LangSmith trace import.

6 Related Work

Flame graphs [11] and pprof [10] establish the folded-stack lineage, where profiles are weighted samples with location hierarchies. Pprof supports sample labels and can visualize them as pseudo stack frames through tag-root/tag-leaf options. Perfetto generalizes trace ingestion with SQL-based analysis and derived events [9]. These systems support query-time aggregation, so AgentProf does not claim novelty for flexible aggregation alone. The difference is structural: pprof's tagroot adds label-derived frames to an existing execution call stack. Agent traces have no execution stack, so the entire profiling structure must be constructed from semantic fields. The evaluation also differs: AgentProf scores profiler output against hidden agent-trajectory labels, a methodology with no analogue in CPU profiling.

OpenTelemetry GenAI and OpenInference standardize GenAI spans [2, 24]. LangSmith [13], Langfuse [14], and Phoenix [1] provide trace visualization, session management, evaluations, and dashboards. AgentOps [8] argues for structured agent observability covering goals, plans, tools, and artifacts. AgentTelemetry [3] defines 14 fault types over 9 agent-specific span kinds that extend OpenTelemetry for agent orchestration phases. Wang et al. [28] survey evidence tracing and execution provenance as foundations for agent accountability. These systems excel at single-execution debugging or fault detection. AgentProf complements them with cross-session aggregate profiling. Datadog LLM Observability [6] and Laminar Signals [12] take steps toward semantic aggregation by clustering user messages or extracting structured events, but they characterize *input distributions*, not *resource attribution.*

AgentRx [4] releases annotated failure trajectories and uses constraint checking plus an LLM judge for root-cause localization. TEL-Bench [26] builds span-localization benchmarks from semantic error annotations. AgentAtlas [22] argues that outcome leaderboards miss trajectory-level quality. TrajAD [17] and AgentFixer [23] detect trajectory anomalies and recommend fixes. These systems perform *span-level* or *step-level* fault localization within a single execution. AgentProf performs *category-level* aggregate profiling, identifying which categories of work produce the most failures, consume the most budget, or generate repeated system effects across sessions.

7 Conclusion

Agent observability needs profiling, not only debugging. AgentProf shows that the semantic operation stack model, combined with pluggable intent attribution and stack construction algorithms, closes the aggregation gap for agent traces. Scoring profiler output against hidden trajectory labels confirms that semantic profiling separates over 90% of mixed system-effect weight, localizes labeled problems at 9.4% inspection work versus flat summaries, and produces labels that agree with ground truth on 7/9 held-out datasets. Fixed-session drilldown remains faster to a first positive on 4/6 tasks, confirming that profilers and debuggers are complementary, and AgentProf supports both by including or excluding session fields in the stack query. Multi-project evaluation and trace-ecosystem integration are the most impactful next steps.

References

[1] Arize AI. 2024. Arize Phoenix. https://arize.com/docs/phoenix.
[2] Arize AI. 2024. OpenInference Specification. https://arize-ai.github.io/openinference/spec/.
[3] Sai Sree Harsha Balusu. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26).* https://dl.acm.org/doi/10.1145/3805760.3814931
[4] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475. doi:10.48550/arXiv.2602.02475
[5] Xinquan Chen et al. 2026. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230. doi:10.48550/arXiv.2605.06230
[6] Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.
[7] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track.*
[8] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285. doi:10.48550/arXiv.2411.05285
[9] Google. 2024. Perfetto Trace Processor. https://perfetto.dev/docs/analysis/trace-processor.
[10] Google. 2024. pprof. https://github.com/google/pprof.
[11] Brendan Gregg. 2011. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.
[12] Laminar. 2025. Signals: Structured Event Extraction from Traces. https://docs.lmnr.ai/signals/introduction.
[13] LangChain. 2024. LangSmith Observability. https://docs.langchain.com/langsmith/observability.
[14] Langfuse. 2024. Langfuse Observability. https://langfuse.com/docs/observability/overview.
[15] Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP).* 3102–3116.
[16] Wei Li, William E. Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. 2024. On the Effects of Data Scale on UI Control Agents. arXiv preprint arXiv:2406.03679. doi:10.48550/arXiv.2406.03679
[17] Yibing Liu, Chong Zhang, Zhongyi Han, Hansong Liu, Yong Wang, Yang Yu, Xiaoyan Wang, and Yilong Yin. 2026. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. *arXiv preprint arXiv:2602.06443* (2026). doi:10.48550/arXiv.2602.06443
[18] Zhaoyang Liu et al. 2025. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. arXiv preprint arXiv:2509.15221. doi:10.48550/arXiv.2509.15221
[19] Quanfeng Lu, Wenqi Shao, Zitao Liu, Fanqing Meng, Boxuan Li, Botian Chen, Siyuan Huang, Kaipeng Zhang, Yu Qiao, and Ping Luo. 2025. GUI Odyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV).*
[20] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. arXiv preprint arXiv:2402.05930. doi:10.48550/arXiv.2402.05930

[21] Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. arXiv preprint arXiv:2504.08942. doi:10.48550/arXiv.2504.08942

[22] Parsa Mazaheri and Kasra Mazaheri. 2026. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv preprint arXiv:2605.20530. doi:10.48550/arXiv.2605.20530

[23] Hadar Mulian, Sergey Zeltyn, Ido Levy, Liane Galanti, Avi Yaeli, and Segev Shlomov. 2026. AgentFixer: From Failure Detection to Fix Recommendations in LLM Agentic Systems. *arXiv preprint arXiv:2603.29848* (2026). doi:10.48550/arXiv.2603.29848

[24] OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. https://github.com/open-telemetry/semantic-conventions-genai.

[25] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerber, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR).*

[26] Jiaming Wang, Ziteng Feng, Jiangtao Wu, Ruihao Li, Qianqian Xie, Yuxiang Ren, He Zhu, Xueming Han, Fanyu Meng, Junlan Feng, and Jiaheng Liu. 2026. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. arXiv preprint arXiv:2606.02060. doi:10.48550/arXiv.2606.02060

[27] Xinyuan Wang et al. 2025. AgentNet (OpenCUA Dataset). https://huggingface.co/datasets/xlangai/AgentNet.

[28] Yiqi Wang, Jiaqi Zhang, Taotao Cai, et al. 2026. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. *arXiv preprint arXiv:2606.04990* (2026).

[29] WukLab. 2025. OSWorld-Human. https://github.com/WukLab/osworld-human.

[30] Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. 2024. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. arXiv preprint arXiv:2412.09605. doi:10.48550/arXiv.2412.09605

[31] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS).*

[32] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS).*

[33] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint arXiv:2406.12045. doi:10.48550/arXiv.2406.12045

[34] Yusheng Zheng, Yanpeng Hu, Tong Yu, and Andi Quinn. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems (PACMI '25, co-located with SOSP).* doi:10.1145/3766882.3767169